Apex Institute of Technology

Computer Science & Engineering

Experiment 9

Name: Trimann Kaur **UID: 24BAI70511**

Branch: B.E. CSE (AIML) **Section: 24AIT_KRG-G1**

Semester: 4 **Date of Performance: 01.04.2026**

Subject Name: Database Management System **Subject Code: 24CSH-298**

AIM: To understand and implement database triggers in PostgreSQL to automate data validation and computational logic, ensuring data integrity by enforcing business rules during DML operations.

OBJECTIVES:

- To understand and implement database triggers in PostgreSQL for automating calculations and enforcing data validation rules, ensuring data integrity during INSERT operations by applying business constraints on employee salary computation.

SOFTWARE REQUIREMENTS:

- Database Management System:
  - PostgreSQL Database
- Database Administration Tool / Client Tool:
  - pgAdmin

PRACTICAL/EXPERIMENT STEPS:

1. An employee table was created to store employee details like ID, name, working hours, salary per hour, and total payable amount.
2. A trigger function CALCULATE_AMOUNT was defined using PL/pgSQL.
3. The function was designed to automatically calculate total payable amount using working hours and salary.
4. A validation rule was added to restrict the total amount to a maximum of 25000.
5. A trigger CAL_PAYABLE_AMOUNT was created to execute before inserting records.
6. A DO block was used to insert a valid record (amount < 25000) for testing.
7. Another DO block was executed to insert an invalid record (amount > 25000) to test exception handling.

Apex Institute of Technology

Computer Science & Engineering

PROCEDURE:

1. The PostgreSQL (pgAdmin) environment was opened and database selected.
2. The employee table was created using CREATE TABLE with required fields and primary key.
3. A trigger function was created using CREATE OR REPLACE FUNCTION in PL/pgSQL.
4. The function logic was written to calculate total amount and enforce the business rule.
5. A trigger was created using CREATE TRIGGER to call the function before INSERT.
6. Test data was inserted using anonymous DO blocks to check both valid and invalid cases.
7. The results were verified using SELECT and error messages (RAISE NOTICE) to confirm correct behavior.

CODE:

```
CREATE TABLE employee (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(50),
   working_hours INT,
   perhour_salary NUMERIC,
   total_payable_amount NUMERIC
);

-- defining trigger function
CREATE OR REPLACE FUNCTION CALCULATE_AMOUNT()
RETURNS TRIGGER
AS
$$
BEGIN
        NEW.total_payable_amount = NEW.perhour_salary * NEW.working_hours;
        IF NEW.total_payable_amount > 25000 THEN
                RAISE EXCEPTION 'AMOUNT IS GREATER THAN 25000';
        END IF;
        RETURN NEW;
END;
$$ LANGUAGE PLPGSQL;

CREATE OR REPLACE TRIGGER CAL_PAYABLE_AMOUNT
BEFORE INSERT
ON employee
FOR EACH ROW
EXECUTE FUNCTION CALCULATE_AMOUNT();
```

```
-- payable amount less than 25000
DO
$$
BEGIN
	INSERT INTO EMPLOYEE(EMP_ID, EMP_NAME, WORKING_HOURS,
perhour_salary) VALUES
	(1, 'Akash', 10, 250);

	EXCEPTION
	WHEN OTHERS THEN
	RAISE NOTICE '%', SQLERRM;
END;
$$

SELECT * FROM EMPLOYEE

-- payable amount more than 25000
DO
$$
BEGIN
	INSERT INTO EMPLOYEE(EMP_ID, EMP_NAME, WORKING_HOURS,
perhour_salary) VALUES
	(2, 'Akash', 10, 25000);

	EXCEPTION
	WHEN OTHERS THEN
	RAISE NOTICE '%', SQLERRM;
END;
$$
```

I/O ANALYSIS:

1. **Creating a Table**
   The employee table is successfully created with fields such as emp_id, emp_name, working_hours, perhour_salary, and total_payable_amount. The primary key constraint ensures unique employee IDs.

```
CREATE TABLE

Query returned successfully in 171 msec.
```

2. **Creating a Function**
   The trigger function CALCULATE_AMOUNT is created successfully. It contains logic to calculate the total payable amount and validate it against the defined limit.

```
CREATE FUNCTION

Query returned successfully in 122 msec.
```

3. **Defining Trigger**
   The trigger CAL_PAYABLE_AMOUNT is successfully defined. It is set to execute before every INSERT operation, ensuring automatic computation and validation of data.

```
CREATE TRIGGER

Query returned successfully in 68 msec.
```

4. **Valid Input**
   When a record is inserted where the total payable amount is within the allowed limit (≤ 25000), the data is successfully inserted, and the computed value is stored in the table.

```
DO

Query returned successfully in 72 msec.
```

	emp_id [PK] integer	emp_name character varying (50)	working_hours integer	perhour_salary numeric	total_payable_amount numeric
1	1	Akash	10	250	2500

5. **Invalid Input**
   When a record is inserted where the total payable amount exceeds 25000, the trigger raises an exception. The insertion is rejected, and an error message is displayed.

```
NOTICE:  AMOUNT IS GREATER THAN 25000
DO

Query returned successfully in 97 msec.
```

Apex Institute of Technology

Computer Science & Engineering

LEARNING OUTCOMES:

1. Understood how to create and use triggers in PostgreSQL to automate operations during database events like INSERT.
2. Learnt to implement data validation and business rules using trigger functions to maintain data integrity.
3. Gained practical knowledge of PL/pgSQL functions and exception handling for controlling and managing database operations effectively.